분야 AI/SW 기초 구분 Linux와 OS 학습시간 40시간 필수/선택 필수

컴퓨터가 갑자기 느려지거나 멈췄을 때 원인 찾아 고치기

문제기술

기술적 설명



1. 미션 소개

Memory Leak, CPU Spike, Deadlock. 이 세 가지 중 하나가 실서버에서 터지면 어떻게 해야 할까요? 로그 없이 재부팅부터 하면 원인이 묻히고, 같은 장애가 두 번, 세 번 반복됩니다. 관제 데이터를 근거로 원인을 추론하고, GitHub Issue 형태의 기술 리포트로 남기는 것까지 직접 해봅니다.

개발자가 작성한 코드는 운영체제 위에서 프로세스 형태로 실행됩니다. 이 미션에서는 빌드된 프로그램을 운영 환경에서 실행하며 발생하는 시스템 장애(Memory Leak/OOM, CPU Spike, Deadlock) 분석을 다룹니다.

단순히 "프로그램이 꺼졌다!"는 결과만 보는 것이 아니라, 관제 데이터와 로그를 통해 장애의 원인을 추론해야 합니다. 이를 바탕으로 현업 개발자처럼 GitHub Issue 형태의 기술 리포트를 작성하며 실전적인 트러블슈팅 및 협업 커뮤니케이션 역량을 기르는 것이 최종 목표입니다.

2. 최종 결과물

다음 과정의 산출물을 PDF 또는 GitHub Repository 링크 형태로 제출해야 한다.

1. 시스템 장애 분석 및 이슈 리포트 (3건)
    - 3가지 장애 유형(OOM Crash, CPU Latency, Deadlock) 각각에 대해 작성된 GitHub Issue 형태의 기술 보고서
    - 각 리포트는 아래의 필수 포함 항목을 모두 갖추어야 한다.

- 발생 현상: 장애가 어떻게 관측되었는지 서술
- 재현 경로 및 증거: 로그/명령어 출력/스크린샷 등 객관적 증거 첨부
- 근본 원인: 장애의 기술적 원인 분석
- 조치 내용: 환경변수 조정 등 임시 조치와 그 결과
- 결과 확인: 조치 후 Before & After 비교 결과

2. 이슈 리포트 마크다운 템플릿

- 각 리포트는 아래 구조를 따르되, 세부 내용은 자유롭게 작성할 수 있다.

```
[Bug] {장애 유형} - {한 줄 요약}

## 1. Description (현상 설명)
- 어떤 현상이 발생했는가?
- 언제, 어떤 조건에서 발생했는가?

## 2. Evidence & Logs (증거 자료)
- monitor.sh 관제 로그 데이터 (수치/그래프/스크린샷)
- 프로그램 실행 로그 중 핵심 구간 발췌
- 시스템 도구(ps, top 등) 출력 결과

## 3. Root Cause Analysis (원인 분석)
- 수집된 증거를 바탕으로 한 기술적 원인 분석
- 관련 OS 동작 원리 설명

## 4. Workaround & Verification (조치 및 검증)
- 어떤 환경변수를 어떻게 조정했는가?
- Before & After 비교 결과 (수치 또는 스크린샷)
- 근본적 해결을 위한 추가 제안 (선택)
```

3. 케이스별 필수 증거 최소 요건

- **OOM**
  - monitor.sh 결과(메모리 상승 수치)
  - 종료 직전/직후 실행 로그("Memory limit exceeded…" , "SELF-TERMINATED…" 등)
  - MEMORY_LIMIT 변경 전후 비교(최소 2회 실행)

- **CPU**
    - CPU 사용률 급상승 구간(top / ps /관제) 캡처
    - 종료 로그("WATCHDOG… SIGTERM" 등)
    - CPU_MAX_OCCUPY 변경 전후 비교
- **Deadlock**
    - PID 존재 증거(ps -ef | grep …)
    - CPU/MEM 변화 정체 증거(top -H 또는 ps -L)
    - 마지막 로그 지점("WAITING… BLOCKED")
    - 스레드/락 대기 추론 근거

3. 미션 목표

해당 미션을 완료한 뒤, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- 메모리 구조를 이해하고, 메모리 누수가 시스템 전체에 미치는 영향을 설명할 수 있다.
- 특정 프로세스의 CPU 과점유가 시스템 지연을 유발하는 원리를 설명할 수 있다.
- 자원 경쟁으로 인해 발생하는 교착상태(Deadlock)의 개념을 이해하고, 프로세스가 멈춘 상태를 시스템 도구로 식별하여 진단할 수 있다.
- 로그와 관제 데이터를 증거로 제시하여 육하원칙에 맞게 장애 상황을 기술하고, GitHub Issue를 통해 동료 개발자와 명확하게 소통할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 사전 준비 사항

- (제공 어플리케이션)
- agent-leak-app을 실행하기 위해서는 아래 조건이 모두 충족되어야 한다.
- 조건이 충족되지 않으면 부트 시퀀스에서 자동으로 실패 처리된다.

항목	조건
실행 계정	root가 아닌 일반 사용자
AGENT_HOME	필수 환경변수 설정
AGENT_PORT	15034 (고정)

항목	조건
AGENT_UPLOAD_DIR	$AGENT_HOME/upload_files (디렉터리 존재 필수)
AGENT_KEY_PATH	$AGENT_HOME/api_keys (경로 존재 필수)
AGENT_LOG_DIR	로그 디렉터리 (존재 + 쓰기 권한)
MEMORY_LIMIT	정수, 50~512 범위 (단위: MB)
CPU_MAX_OCCUPY	정수, 10~100 범위 (단위: %)
MULTI_THREAD_ENABLE	true / false (1 / 0 , yes / no 허용)
secret.key 파일	$AGENT_HOME/api_keys/secret.key 존재, 내용: agent_api_key_test
네트워크	0.0.0.0:15034 바인딩 가능

2. 메모리 누수 원인 규명 및 리포팅

- monitor.sh 를 활용하여, 대상 프로세스(agent-leak-app)의 물리 메모리 사용량이 시간 경과에 따라 증가하는 패턴을 관측한다.
- 프로세스가 예고 없이 중단되었을 때 프로그램 실행 로그를 분석하여 메모리 임계치 초과로 인해 애플리케이션의 메모리 보호 정책(MemoryGuard)에 따라 강제 종료되었음을 나타내는 핵심 로그를 식별한다.
- 환경변수(MEMORY_LIMIT)를 조정하여 프로그램이 더 오래 생존하는 것을 확인하고, 그 결과를 리포트에 Before & After로 기록한다.

3. CPU 과점유 분석 및 리포팅

- 관제 툴과 로그를 통해 시스템 전체 부하가 아닌 특정 프로세스(agent-leak-app)의 CPU 사용률이 급격히 상승하는 구간을 식별한다.
- 프로그램 실행 로그 분석을 통해, 해당 종료가 오류가 아닌 과점유 방지 정책(Watchdog)에 따른 시스템 보호 조치였음을 입증한다.
- 환경변수(CPU_MAX_OCCUPY)를 조정하여 프로세스 종료 여부 또는 생존 시간 변화를 확인하고, 그 결과를 리포트에 Before & After로 기록한다.

4. 교착상태(DeadLock) 진단 및 리포팅

- 프로세스가 종료되지 않고 살아있으나(PID 존재), CPU/메모리 변화가 없고 로그 기록도 멈춘 무응답 상태임을 식별한다.
- 프로그램 로그의 마지막 기록을 분석하여, 서로 다른 스레드가 상대방의 자원을 무한히 기다리는 상태임을 논리적으로 증명한다.
- 환경변수(MULTI_THREAD_ENABLE)를 조정하여 데드락 재현/회피 비교 결과를 리포트에 기록한다.
- Deadlock(교착상태)의 개념이 생소한 학습자는 아래 키워드를 참고하여 기본 개념을 학습한 후 미션에 임하는 것을 권장한다.
    - 식사하는 철학자들 문제(Dining Philosophers Problem): 교착상태의 대표적인 비유 모델
    - 교착상태 4대 조건: 상호 배제(Mutual Exclusion), 점유 대기(Hold and Wait), 비선점(No Preemption), 순환 대기(Circular Wait)

5. 보너스 과제 (선택)

- 스케줄링 알고리즘 추론
    - 프로그램 로그 데이터 분석: 로그의 타임스탬프를 기반으로 프로세스 간 실행 순서와 교체 주기를 패턴화한다.
    - 알고리즘 역추론: 도출된 패턴을 근거로, 현재 프로그램에 적용된 스케줄링 기법이 Round-Robin, FCFS, Priority 중 무엇인지 논리적으로 추론한다.
    - 장단점 및 적합한 아키텍처 분석: 추론한 스케줄링 알고리즘의 기술적 장단점을 서술하고 어떤 성격의 서비스(예: 실시간 응답이 중요한 웹 서버 vs 처리량이 중요한 배치 서버 등)에 적합한지 분석하여 정리한다.
    - 스케줄링 추론 리포트 예시

```
CSS
# [Analysis] 로그 패턴 분석을 통한 스케줄링 알고리즘 추론

## 1. 로그 관찰 개요
`agent-leak-app`의 정상 실행 상태에서 발생하는 워커(Worker) 스레드들의 작업 로그를 수집하여, OS 또는 런타임이 작업을 처리하는 스케줄링 기법을 역추적했습니다.

## 2. 증거 자료
로그의 타임스탬프와 작업 진행률(Progress)을 분석한 결과, 하나의 작업이 완료되기 전에 다른 작업이 끼어드는 현상이 관측되었습니다.

[ Application Log Snapshot ]
```

[2025-12-30 14:00:00.100] [Thread-A] Task Started. Calculating... (10%)
[2025-12-30 14:00:00.150] [Thread-A] Calculating... (20%)
[2025-12-30 14:00:00.200] [Thread-B] Task Started. Calculating... (10%) <-- A 중단, B 시작
[2025-12-30 14:00:00.250] [Thread-B] Calculating... (20%)
[2025-12-30 14:00:00.300] [Thread-C] Task Started. Calculating... (10%) <-- B 중단, C 시작
[2025-12-30 14:00:00.350] [Thread-A] Resumed. Calculating... (30%) <-- C 중단, A 재개

3. 패턴 분석 및 결론
* 순차 처리 아님: Thread-A가 100% 완료되기 전에 Thread-B가 실행되었으므로, 먼저 온 작업을 끝까지 처리하는 방식이 아닙니다.

* 우선순위 아님: 특정 스레드가 자원을 독점하거나, 긴급하게 처리되는 경향 없이 A, B, C가 공평하게 나눠 가지는 모습을 보입니다.

* 최종 결론: 각 스레드가 정해진 시간 할당량만큼 CPU를 사용하고, 자원을 반납하는 라운드 로빈 알고리즘으로 추론됩니다.

개발환경

6. 개발 환경

- 제공된 바이너리(Python 기반)를 실행할 수 있는 리눅스 환경
- 로컬 또는 격리된 환경(Docker 컨테이너 등)에서 실행을 권장
- 공유 네트워크 환경에서는 방화벽 설정에 유의
- 바이너리 디컴파일 및 리버스 엔지니어링 시도 금지

제약조건

7. 제약 사항

- monitor.sh , ps , top , htop , pstree , kill 등 리눅스 표준 명령어 및 라이브러리 사용

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다.실제 문구와 디자인은 달라도 된다.

- 장애 분석 리포트 예시 (GitHub Issue - OOM 분석 Case)

CSS

[Bug] 프로세스 실행 10분 후 메모리 보호 정책에 의한 비정상 강제 종료

1. Description (현상 설명)
`agent-leak-app` 어플리케이션을 실행하고 약 10분이 경과하면, 터미널에 `SELF-TERMINATED` 메시지가 출력되며 프로세스가 예고 없이 종료됩니다. 애플리케이션 내부의 메모리 보호 정책(MemoryGuard)에 의해 프로세스가 강제 종료되는 현상이 반복됩니다.

2. Evidence & Logs (증거 자료)
`monitor.sh`를 통해 수집된 관제 로그를 분석한 결과, **메모리 점유율(MEM)**이 초기 5%대에서 시작하여 종료 직전 96%까지 **선형적으로 급격히 상승**하는 패턴이 확인되었습니다. 반면 CPU 사용률은 안정적이었습니다.

[monitor.log 데이터 발췌]
[2025-12-30 14:00:00] PROCESS:agent-leak-app CPU:1.2% MEM:5.1% DISK:954G FIREWALL:active
[2025-12-30 14:03:00] PROCESS:agent-leak-app CPU:1.5% MEM:35.4% DISK:954G FIREWALL:active
[2025-12-30 14:06:00] PROCESS:agent-leak-app CPU:1.4% MEM:68.2% DISK:954G FIREWALL:active
[2025-12-30 14:09:00] PROCESS:agent-leak-app CPU:1.3% MEM:89.5% DISK:954G FIREWALL:active
[2025-12-30 14:10:00] PROCESS:agent-leak-app CPU:1.5% MEM:96.8% DISK:954G FIREWALL:active

[프로그램 실행 로그 발췌]
[CRITICAL] [MemoryGuard] Memory limit exceeded (256MB >= 256MB) /

(Recommend Over 256MB)
[CRITICAL] [MemoryGuard] Self-terminating process 12345 to prevent system instability.
>>> [SYSTEM] SELF-TERMINATED (Memory Limit Exceeded) <<<

3. Root Cause Analysis (원인 분석)
현상 분석: 어플리케이션 로직 내부에서 생성한 데이터를 힙(Heap) 메모리에서 해제하지 않고 지속적으로 쌓는 메모리 누수(Memory Leak) 결함이 있는 것으로 판단됩니다.
시스템 동작: 물리 메모리 사용량이 MEMORY_LIMIT에 도달하자, 애플리케이션 내부의 **MemoryGuard 정책**이 시스템 전체 불안정을 방지하기 위해 해당 프로세스를 SIGKILL로 강제 종료시켰습니다.

4. Workaround & Verification (조치 및 검증)
조치 내용: .bash_profile 내의 환경변수 MEMORY_LIMIT 값을 기존 256MB에서 512MB로 상향 조정하여 임시적으로 가용 메모리를 확보했습니다.
검증 결과: 설정 변경 후 재실행 결과, 기존 종료 시점인 10분을 넘겨 30분 이상 프로세스가 생존함을 확인했습니다. 다만, 근본적인 해결을 위해서는 소스 코드 내 불필요한 데이터를 주기적으로 삭제(del or pop)하는 리팩토링이 필요합니다.

- 장애 분석 리포트 구조 예시 (GitHub Issue - CPU 과점유 Case, 정답 아님)

```
CSS
[Bug] CPU 과점유에 의한 Watchdog 보호 조치 프로세스 종료

## 1. Description (현상 설명)
`agent-leak-app` 실행 후 일정 시간이 경과하면, CPU 사용률이 급격히 상승하고 "[SYSTEM] WATCHDOG: INITIATING EMERGENCY ABORT (SIGTERM)" 메시지와 함께
프로세스가 종료됩니다.

## 2. Evidence & Logs (증거 자료)
- monitor.sh 관제 로그에서 CPU 사용률 변화 추이 (캡처/수치 첨부)
- 프로그램 실행 로그에서 Watchdog 관련 로그 발췌
- top 또는 ps 명령어를 통한 프로세스별 CPU 점유율 확인 결과

## 3. Root Cause Analysis (원인 분석)
- CPU 부하가 내부 Watchdog 임계치를 초과한 원인 분석
- 과점유 방지 정책의 동작 원리 서술
```

```
## 4. Workaround & Verification (조치 및 검증)
- CPU_MAX_OCCUPY 환경변수 조정 전후 비교 (Before & After)
- 조정 후 프로세스 생존 시간 또는 종료 여부 변화 기록
```

- 장애 분석 리포트 구조 예시 (GitHub Issue - Deadlock Case, 정답 아님)

```
CSS
[Bug] 멀티스레드 환경에서 교착상태(Deadlock) 발생으로 프로세스 무응답

## 1. Description (현상 설명)
`agent-leak-app` 실행 후 프로세스가 종료되지 않고 PID가 유지되나,
CPU/메모리 변화가 없고 로그 출력도 완전히 멈춘 무응답 상태가 지속됩니다.

## 2. Evidence & Logs (증거 자료)
- ps -ef | grep agent 결과 (PID 존재 확인)
- top -H 또는 ps -L 결과 (스레드별 CPU/MEM 변화 없음 확인)
- 프로그램 실행 로그의 마지막 기록 발췌 (WAITING/BLOCKED 로그)

## 3. Root Cause Analysis (원인 분석)
- 마지막 로그를 근거로 스레드 간 순환 자원 대기 상태 추론
- 상호 배제와 순환 대기 원리 서술

## 4. Workaround & Verification (조치 및 검증)
- MULTI_THREAD_ENABLE 환경변수를 false로 변경하여 재실행
- 데드락 발생 여부 비교 (Before: true → Deadlock / After: false → 정상 동작)
```

데이터

데이터파일

agent-app-leak.zip

데이터파일설명

agent-app-leak-x86 (Intel chip) agent-app-leak-arm64 (Apple chip)